DECISION MODELS · GPU WORKSHOP

문장을 생성하는 대신 결정을 학습한다

Jev · Laya · Clef · Unsloth

상태와 질문을 입력하고, 선택지의 확률을 얻는 모델

> 문의마다 담당 팀을 골라야 하고, 그 판단을 **얼마나 믿을 수 있는지**도 알아야 한다면?

딥러닝 경험자 · NVIDIA GPU 서버 · 110분
2026.10.13 · 개념 / 학습 / 보정 / 서빙

노트북 세 개로 결과물을 만든다

01

확률을 읽는다

정확도 · NLL · Brier · ECE
보정 온도와 보류(사람 검토) 임계값

`01_probabilities…ipynb`

02

모델을 학습한다

Qwen + 결정 헤드 + LoRA
보정 · 저장 · 재로딩

`02_train…ipynb`

03

API를 검증한다

응답 형식 규칙(계약)
정확도 · 지연시간 · 보류

`03_test…ipynb`

완료 기준: 정답을 고르는 것과 그 확률을 믿을 수 있는 것을 **따로 측정**한다.

핵심 주의 · 하지 말 것

끝나면 네 가지를 할 수 있다

#	학습 목표	확인 방법
1	결정 모델의 입력(질문·선택지 정의)과 타입별 출력을 **해석**하고 생성형 LLM과 구분한다	마무리 Q1
2	정답률과 확률 품질을 따로 **측정**하고 온도·임계값을 선택한다	노트북 01 · Q2, Q3
3	누수 없는 분할로 결정 모델을 학습·보정하고 재로딩을 **검증**한다	노트북 02 · Q5
4	로컬 API를 응답 형식·연결·품질로 **나눠 테스트**한다	노트북 03 · Q4

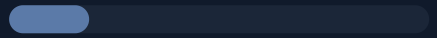
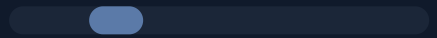
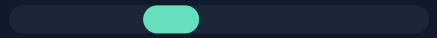
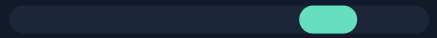
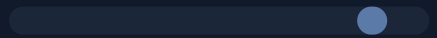
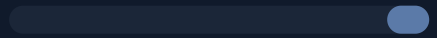
> 각 목표는 마무리 질문과 노트북 산출물로 확인한다.

시작 전 예측: 세 문항

1. 모델이 확률 **0.9**로 답한 사례들의 실제 정답률은 90%일까?
2. 학습 전 정확도가 낮다면 Qwen 본체가 약하다는 뜻일까?
3. 학습이 끝난 뒤 확률을 조정하는 **보정 온도**를 바꾸면 **accuracy**도 바뀔까? (생성 샘플링 온도와 다른 개념)

> 답은 적어 두기만 한다. 마무리에서 같은 문항에 다시 답하고 바뀐 답을 비교한다.

질문 하나씩 답하며 110분을 진행한다

시간	주제	0 — 110분	확인할 질문
0–21분	Jev와 질문 스키마		무엇을 출력하는가?
21–35분	구조와 손실		무엇을 학습하는가?
35–50분	확률·평가 실습		확률을 믿어도 되는가?
50–76분	GPU 학습 실습		데이터가 새지 않았는가?
76–91분	저장·서빙·API 테스트		같은 모델이 연결되었는가?
91–99분	실험 설계와 마무리		결과가 어디까지 유효한가?
99–110분	질의응답 · 여유		

개념 실습 GPU 실습에는 **학습 대기 8분**(실측 전 계획값)이 들어 있고, 대기 중 수치 해석 두 장을 진행한다.

구조 설명 뒤 심화 Q&A 14장은 선택 자료다. 모두 설명하면 기본 110분 외에 약 16분을 추가한다.

PART 01 · INTERFACE

결정 모델은 무엇을 입력받고 무엇을 내놓는가?

상태 + 질문 스키마 → 선택지 확률

질문은 유연하게, 응답은 확률로

접근	답을 얻는 방법	비용과 제약
LLM	문장·JSON을 생성한 뒤 파싱	토큰 생성 시간 · 형식 오류 처리 선택지 확률은 별도 설계 필요
BERT 분류기	작업별로 정해진 라벨을 분류	출력 라벨이 바뀌면 헤드 수정·재학습 필요
Jev	요청마다 질문·선택지를 정의 **선택값과 확률을 구조화해 반환**	새 질문·선택지도 입력으로 전달 실제 성능과 확률 보정은 검증 필요

> **Jev의 선택:** 질문·선택지를 입력으로 받는 유연성과
> 소프트웨어가 바로 사용할 수 있는 확률 출력을 결합한다.

참고: PyTorch 한국 사용자 모임 · Jev와 System One 소개

생성은 문장, 결정은 분포

생성형 LLM

```
입력: 고객이 중복 결제를 문의함
출력: "billing 팀으로 보내세요."
```

- 문자열 또는 JSON 생성
- 생성 토큰과 형식 처리 필요
- 선택지 확률은 별도 설계 필요

결정 모델

```
{
  "billing": 0.80,
  "technical": 0.15,
  "sales": 0.05
}
```

- 입력: 상태 + 질문 “담당 팀은?” + 선택지 3개
- **확률을 직접 후속 정책에 사용**

logprob 분류와 무엇이 다르고, 언제 쓰지 않는가

방식	선택지 확률을 얻는 법	주의할 점
생성 + 파싱	따로 없음	형식 오류 처리, 확률은 별도 설계
LLM logprob	선택지 토큰의 확률로 근사	선택지가 여러 토큰이면 선택지마다 채점, 과제에 맞춘 보정 없음
결정 모델	선택지 전체 분포를 직접 출력	헤드 학습과 보정이 필요

쓰기 좋은 곳

선택지가 정해진 분류 · 라우팅 · 가드레일
확률로 자동 처리 범위를 정할 때 · 대량 요청

쓰지 않는 곳

자유 서술 답변 · 설명 자체가 결과물일 때
선택지를 미리 정할 수 없는 과제

Jev는 개념의 출발점, 학습 대상은 공개 모델

System One: Kahneman의 빠른 직관적 사고(System 1)에서 따온 이름. 자유 생성 대신 구조화된 결정을 빠르게 내는 모델 계열이다.

구분	역할	이 강의에서의 사용
Jev	TypeSafe의 System One API 모델 (구조·가중치 비공개)	따라 할 API 형식의 기준
Laya	인코더 기반 공개 결정 모델	CPU 로컬 API 테스트 경로
Clef	LLM 본체 + 결정 헤드의 공개 모델	구조적 참고
Unsloth 변환 모델	공개 LLM에 결정 헤드를 붙여 학습	Qwen + LoRA 실습

> API 형식이 같아도 **가중치·확률 보정·정확도·지연시간은 다르다.** 로컬 Studio의 `jev-latest` 는 **Settings에서 고른 모델의 별칭**이며 TypeSafe를 호출하지 않는다.

출처: TypeSafe 소개 · Unsloth 로컬 서빙

요청은 State + Questions, 응답은 타입별 확률

State: 판단할 텍스트·JSON | **Questions**: 그 상태에 대해 답할 항목들

유형	용도	돌려받는 값
`noul`	예 / 아니오	"예"일 확률 · 0~1
`choice`	여러 선택지 중 하나 고르기	가장 유력한 선택지 + 선택지별 확률
`score`	순서가 있는 등급 매기기	등급별 확률 + 기대 점수 · 등급 번호는 0부터

choice·score의 confidence: 1에 가까우면 한 답이 두드러지고,
0에 가까우면 확률이 고르게 퍼진다. 실제 정답률을 뜻하지 않는다.

실제 동작은 선택지별 probabilities로 결정한다.
예: `probabilities["technical"] > 0.8` 일 때만 버그 리포트 생성

Unsloth Decision API 문서 기준: 요청당 **64개 질문** · choice당 **255개 선택지** · score당 **10개 등급**
출처: Unsloth · Laya & Decision Models

하나의 상태에 여러 질문을 함께 묻는다

```
state = "어제 두 번 결제됐어요. 오늘 중복 금액을 환불해 주세요."
questions = {
    "team": {
        "type": "choice", "instructions": "담당 팀은?",
        "criteria": {"billing": "결제·환불", "technical": "오류", "sales": "견적"}
    },
    "refund": {"type": "noul", "instructions": "환불을 요청하는가?"},
    "urgency": {
        "type": "score", "instructions": "처리 긴급도는?",
        "criteria": ["일반", "곧 처리", "오늘 처리"]
    }
}
```

`noul` 예/아니오 · criteria 없음 | `choice` 하나 고르기 · `{키: 설명}` | `score` 순서 등급 · 낮은 것부터 목록(번호 0, 1, 2)

질문·선택지 설명도 모델 입력이다. `state` 는 문자열 또는 JSON.

응답은 질문 키마다 타입별 필드를 돌려준다

```
{
  "answers": {
    "team":    { "type": "choice", "choice": "billing", "confidence": 0.80,
                 "probabilities": { "billing": 0.80, "technical": 0.15, "sales": 0.05 } },
    "refund":  { "type": "noul", "noul": 0.90 },
    "urgency": { "type": "score", "score": 1.6, "confidence": 0.70,
                 "legend": { "0": "일반", "1": "곧 처리", "2": "오늘 처리" },
                 "probabilities": { "0": 0.1, "1": 0.2, "2": 0.7 } }
  }
}
```

- 질문 키가 그대로 응답 키다. `"choice": "billing"` 은 **고른 선택지**이며 `type` 이름과 같을 뿐이다.
- `confidence` **는 확률 분포의 집중도를 나타내는 값**이며 계산은 모델마다 다르다. `legend` 는 등급 번호의 이름표다.

세 타입의 의미를 구분하자

타입	응답 핵심	해석	흔한 실수
`noul` (예/아니오)	`noul: 0.8`	yes 확률 80%	boolean으로 간주
`choice`	선택 키 + 확률 분포	가장 높은 선택지	confidence를 정답률로 해석
`score`	등급 확률 + 기대 점수	등급 번호의 기댓값	argmax 등급과 혼동

$$\text{score} = \sum_{k=0}^{K-1} k\, p_k$$

1.6
기대 점수 score

2
확률이 가장 높은 등급 argmax

등급 `[0, 1, 2]` 의 확률이 `[0.1, 0.2, 0.7]` 일 때
score = 0 × 0.1 + 1 × 0.2 + 2 × 0.7 = **1.6**
score는 등급에 확률을 곱해 더한 평균,
argmax는 확률이 가장 높은 등급이다.

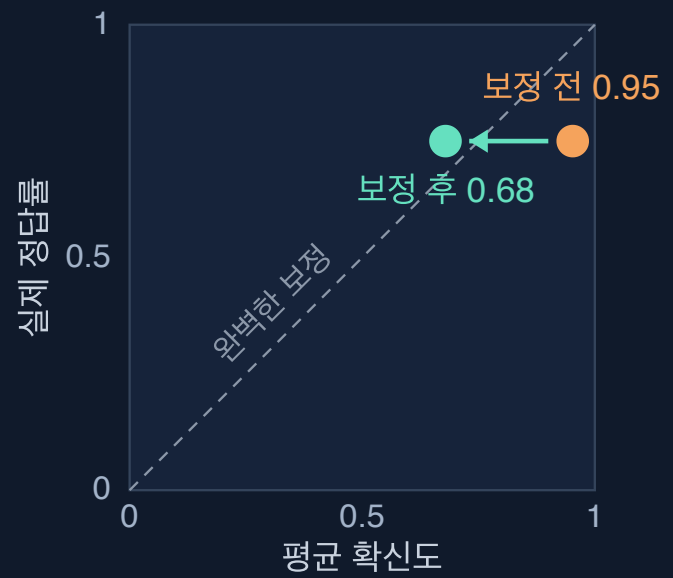
확률과 confidence는 같은 말이 아니다

모델 출력

고른 답의 확률(confidence) 0.9는 정답률 90%를 **보장하지 않는다.**

검증할 성질

0.9로 예측한 사례 중 실제로 약 90%가 정답인가? 이 관계가 **calibration(보정)**이다.



모델을 바꿀 때 임계값도 validation에서 **다시 선택**한다.

서빙 문서: Laya와 Jev는 confidence 계산이 달라 Jev의 임계값을 재사용하지 말고, choice는 probabilities에 임계값을 건다. 문서

PART 02 · MODEL

어떤 파라미터가 결정을 배우는가?

공개 LLM 본체 + 질문을 읽는 결정 헤드

LLM은 한 번 읽고, 결정 헤드는 함께 채점한다

하나의 프롬프트 = State + 모든 질문·선택지

단계	Unsloth의 Clef식 결정 모델이 하는 일
LLM 본체	순전파 1회로 마지막 층의 은닉 상태를 계산
결정 헤드	질문·선택지 구간의 표현을 읽어 모든 질문을 함께 처리하고 채점
확률 출력	다음 토큰을 생성하지 않고 선택지별 확률을 반환

텍스트 decoding 없음: 서빙 예시의 `output_tokens: 0`
어휘 전체의 다음 토큰 logits 계산을 생략하지만, 입력 처리·은닉 상태 메모리는 필요하다.

새 헤드의 기본 구성

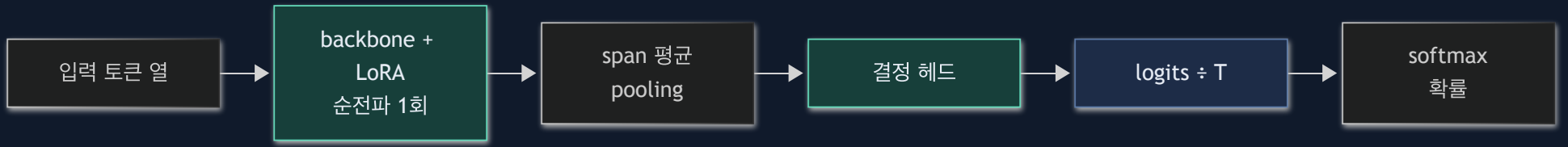
라우팅 2층 + 디코더 4층
본체 은닉 차원 ≥ 3072: 폭 1024 · 16 heads
본체 은닉 차원 < 3072: 폭 512 · 8 heads

학습·저장

일반 LLM에 결정 헤드를 새로 초기화
본체 LoRA + 헤드를 함께 학습
헤드 학습률 기본값: `1e-4`

출처: Unsloth 학습 가이드 · decision_from_lm.py (고정 커밋) · 서빙 예시

한 번의 본체 순전파로 여러 질문에 답한다



학습: LoRA · 헤드　보정 단계에서 결정: 온도 T　나머지 본체 가중치는 고정

- 입력은 state·질문·선택지를 이은 **하나의 토큰 열**이다. 모든 질문이 같은 순전파를 공유한다.
- 질문·선택지마다 **토큰 구간(span)의 평균**을 내고, 마지막 토큰은 입력 전체를 요약한 전역 벡터로 쓴다.
- 헤드는 질문들을 **한 번에 함께 처리**(attention)하고, 질문 벡터와 각 선택지 벡터의 **유사도**로 선택지 점수를 낸다. 처음 보는 선택지도 채점할 수 있는 이유다.
- 문장 decoding은 없지만 긴 입력을 읽는 비용은 남는다(`output_tokens=0` ≠ 비용 0).

Unsloth의 Clef식 구조이며 Jev 내부 구조는 추정하지 않는다. 구현: decision_from_lm.py · clef.py (batched_logits)

LLM → 결정 모델: 구조를 붙이고 판단을 학습한다

단계	코드·동작	이때의 의미
1. 본체 로딩	사전학습된 Qwen 등의 가중치 로딩	언어·문맥 표현을 재사용
2. 헤드 추가	`JointSchemaHead(**config)`	새 헤드는 무작위 초기화
3. 모델 연결	`ClefDecisionModel(backbone, head)`	본체 표현 → 선택지 점수
4. 학습	`get_peft_model` → `DecisionTrainer.train()`	정답으로 LoRA와 헤드를 조정
5. 저장	`model.save_pretrained(...)`	학습한 LoRA·헤드를 보관

헤드를 붙인 직후 ≠ 판단 학습 완료.
상황·질문·선택지와 정답을 함께 제공해야 채점 능력을 배운다.

소스: load_lm_as_decision_model · 실습: `02_train_decision_model.ipynb` §4·6·8

새 질문을 입력할 수 있다 ≠ 새 질문도 잘 맞힌다

공유 헤드가 질문·선택지의 의미를 읽는다. 질문마다 별도 분류기를 만들지 않는다.

질문의 변화	기대와 검증
"담당 팀은?" → "어느 부서로 전달할까?"	의미가 같은 표현 변화 → 일반화 가능, 평가로 확인
새 팀 설명·새 선택지 추가	구조 변경 없이 채점 가능 → 선택 정확도는 별도 검증
담당 팀 분류 → 긴급도 판단	다른 판단 기준 → 추가 학습이 필요할 수 있음
고객 문의 → 의료 판단	영역·지식의 변화 → 기존 성능을 그대로 기대할 수 없음

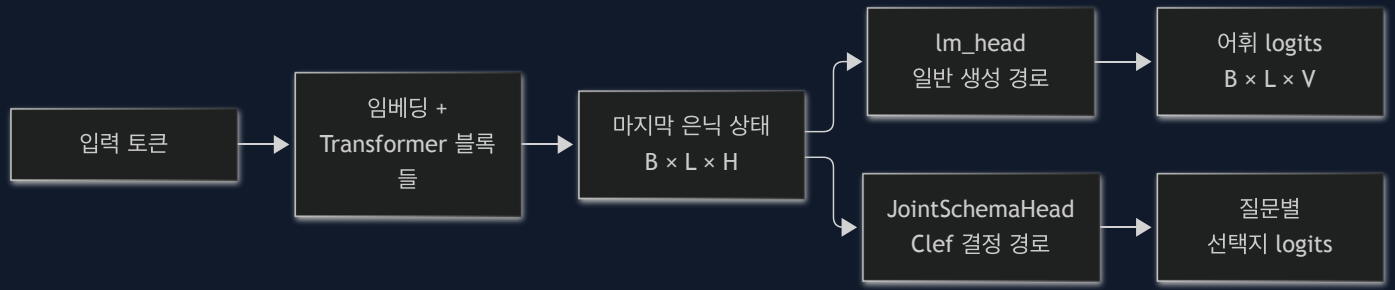
왜 일반화가 가능한가?

본체의 사전학습 표현 + 다양한 질문으로 학습한 공유 채점 방식

어떻게 확인하는가?

새 사례 평가에 더해, 질문 종류 자체를 학습에서 제외한 평가

마지막 은닉 상태는 lm_head의 출력이 아니다



- `L` : 입력 토큰 수 · `H` : 본체 은닉 차원 · `V` : 어휘 수 · `B` : 배치 크기
- **마지막 층의 모든 토큰 표현**을 전달한다. 마지막 토큰 하나만 쓰는 것이 아니다.
- Clef는 어휘 전체 logits를 계산하지 않지만, **출력 임베딩 가중치**는 선택지 표현에 사용한다.

소스: ClefDecisionModel.forward

JointSchemaHead 안에도 Transformer가 있다

구성	입력·처리	역할
표현 준비	정규화 · span 평균 · `H → W` 투영	질문·선택지·전체 토큰 메모리 구성
근거 라우팅	선택지 query → 전체 메모리 cross-attention	선택지마다 상황의 근거를 반영
공동 질문 디코더	질문 간 self-attention + 메모리 cross-attention	여러 질문의 표현을 함께 갱신
채점	어휘 prior + cosine + 잔차 MLP	질문별 선택지 logits 계산

질문별 벡터는 유지된다. 함께 attention해도 질문들이 하나의 결과로 합쳐지는 것은 아니다.

새 헤드 기본값: 라우팅 2층 · 디코더 4층 · 폭 W=1024 또는 512.
여기서 "디코더"는 질문 벡터를 처리하며 답변 문장을 생성하지 않는다.

소스: 헤드 모듈 정의 · 계산 경로

중간 Transformer 층이 더 좋을 수도 있다

마지막 층은 다음 토큰 예측에 적합하게 학습된다. 결정 작업의 최적 층이라는 보장은 없다.

조건	마지막 층 사용의 의미
본체 고정 + 헤드 학습	기존 표현에서 판단 정보를 읽는 방법을 배움
본체 LoRA + 헤드 학습	본체 표현도 결정 손실에 맞춰 적응 가능

비교 실험	확인할 가설
마지막 층 / 중간·후반 층 하나 / 여러 층의 가중합	어느 표현이 판단에 더 적합한가?

연결만 바꾸면 공정한 비교가 아니다.
각 표현에 맞춰 헤드를 다시 학습하고, 같은 분할·설정으로 평가한다.

연구: Layer by Layer (2025) — 텍스트 임베딩 과제에서 중간 층의 이점 보고.
Clef에서 중간 층이 더 좋다는 직접 비교 근거로 해석하지 않는다.

API 입력: 상황과 질문·선택지를 JSON으로 준다

```
{
  "model": "선택한 모델 ID",
  "state": "중복 결제됐어요. 환불해 주세요.",
  "questions": {
    "team": {
      "type": "choice",
      "instructions": "담당 팀을 선택하세요.",
      "criteria": {
        "billing": "결제 오류 및 환불",
        "technical": "서비스 장애 및 오류"
      }
    },
    "refund": {
      "type": "noul",
      "instructions": "환불 요청인가요?"
    }
  }
}
```

- `state` : 판단할 상황. 문자열 또는 JSON 값
- `team` , `refund` : 결과를 연결할 **질문 ID**
- `instructions` : 판단 기준·질문 문구
- `criteria` : 선택지 ID와 의미 설명
- `noul` : 기본 선택지는 **true / false**

> 선택지의 설명도 입력이다.
> 고정된 클래스 번호만 채점하는 구조와 다르다.

백본 입력: 질문·선택지까지 하나의 텍스트로 읽는다

```
<|im_start|>system
Read the complete state and schema. ...
<|im_end|>
<|im_start|>user
STATE:
중복 결제됐어요. 환불해 주세요.
SCHEMA FIELDS:
FIELD 1
ID: team
TYPE: choice
INSTRUCTION: 담당 팀을 선택하세요.
ALLOWED OPTIONS:
OPTION 1: {"option_id":"billing", ...}
OPTION 2: {"option_id":"technical", ...}
END FIELD
... refund 질문 ...
<|im_end|>
<|im_start|>assistant
<think> ... </think>
JOINT SCHEMA DECISIONS:
```

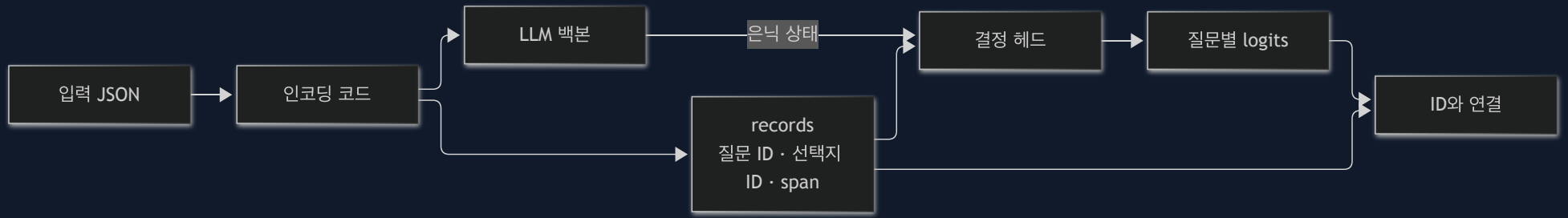
소스: encode_record

실제 형식의 일부 생략

- 특수 토큰을 포함해 토큰화
- 선택지에는 ID와 설명을 함께 포함
- 마지막 표식까지가 **입력**
- 이후 답변 문장을 생성하지 않고 **은닉 상태를 헤드로 전달**

사용자는 JSON만 전달한다.
이 텍스트와 구간 정보는 인코딩 코드가 만든다.

필드와 선택지는 span 메타데이터로 추적한다




```
# 구조를 단순화한 예시: 실제 토큰 위치 값은 인코딩 시 계산
EncodedQuestion(
    question_id="team",
    question_span=(q_start, q_end),
    option_spans=((b_start, b_end), (t_start, t_end)),
    option_ids=("billing", "technical"),
)
```

모델이 필드 이름이나 구간을 추측하지 않는다. **프로그램이 기록한 연결 관계**를 사용한다.

헤드는 질문마다 선택지 수만큼 숫자를 출력한다

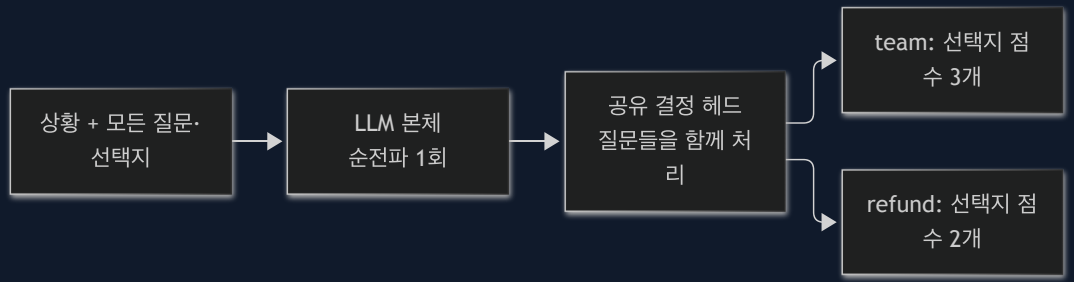
단계	한 요청의 형태	예시
백본 입력	`input_ids: [1, L]`	상황 + 질문 2개 + 선택지 4개
백본 출력	`hidden: [1, L, H]`	입력 토큰별 문맥 벡터
헤드의 질문 표현	`[2, W]`	team, refund 벡터를 각각 유지
헤드의 선택지 표현	`[4, W]`	billing, technical, true, false
헤드 출력	질문마다 길이가 다른 점수 벡터 가능	아래 logits 목록

```
# 설명용 점수. 같은 채점 함수를 모든 질문·선택지에 공유한다.
logits = [
    [3.0, 1.0],  # team   → billing, technical
    [2.0, 0.0],  # refund → true, false
]
```

> 질문별 독립 헤드가 아니라 **공유 헤드 + 질문별 벡터 + 입력 선택지**다.
> 배치에서는 요청별 → 질문별 → 선택지별 점수 구조로 반환한다.

LLM은 한 번 실행하고, 질문별 결과를 만든다

한 요청: "중복 결제됐어요. 환불해 주세요." + 아래 질문 두 개



질문	이번 입력의 선택지	최종 채점
담당 팀은?	결제팀 · 기술팀 · 영업팀	질문 표현과 선택지 3개를 비교
환불 요청인가?	예 · 아니오	질문 표현과 선택지 2개를 비교

본체 계산은 공유하고, 결과는 질문별로 구분한다.
같은 채점 함수를 묶어서 병렬 적용할 수 있다. 질문마다 본체를 재실행하지 않는다.

고정인 것은 벡터 차원, 가변인 것은 선택지 수다

달라질 수 있는 것	처리 방식	결과
선택지 문구의 토큰 수	구간의 은닉 상태를 평균하고 투영	선택지마다 고정 차원 `W`
질문의 선택지 수 `K`	같은 채점 함수를 선택지마다 적용	점수 `K` 개

```
# 원리 설명용: 실제 Clef는 질문·선택지 관계도 함께 사용한다.
scorer = nn.Linear(W, 1)  # 파라미터 크기는 고정

# 각 행이 선택지 하나. 마지막 차원 W만 1로 변환한다.
scorer(options_team)    # [3, W] → [3, 1]
scorer(options_refund)  # [2, W] → [2, 1]
```

선택지 표현

선택지 3개면 벡터 3개
각 벡터의 차원 W는 동일

선택지 점수

각 행에 점수 하나씩
출력 뉴런 수를 바꾸지 않음

실제 구현: 선택지를 묶고, 개수대로 나눈다

```python
# forward_per_record 계산 흐름의 핵심 (W: 헤드 폭)
option_counts = [len(q.option_spans) for q in record.questions]
# 이번 예시: [3, 2]

# 라우팅을 마친 선택지 표현: routed[0].shape == [5, W]
split_options = torch.split(routed[0], option_counts, dim=0)
# 질문 0: [3, W] / 질문 1: [2, W]

# 공유 채점기로 질문 벡터와 각 선택지의 관계를 계산
# 질문 0: 특징 [3, 4W] → MLP [3, 1] → 점수 [3]
# 질문 1: 특징 [2, 4W] → MLP [2, 1] → 점수 [2]
```

- `owner = [0, 0, 0, 1, 1]` 로 각 선택지에 **소속 질문의 표현**을 연결한다.
- MLP 점수에 어휘 prior·cosine 점수를 결합하고, **질문별 벡터 목록**으로 반환한다.
- 기본 fast 경로는 **패딩 + 마스크 + 인덱싱**으로 묶어 계산한 뒤 유효 선택지만 반환한다.

소스: 선택지 분리·공유 채점 · batched_logits · ClefLogits

질문별 확률과 타입별 응답은 코드로 계산한다

각 질문 안에서 softmax를 적용한다. 전체 질문의 선택지를 한꺼번에 정규화하지 않는다.

질문	logits (설명용)	확률	반환할 판단
team	`[3, 1]`	billing 0.8808 · technical 0.1192	`choice: "billing"`
refund	`[2, 0]`	true 0.8808 · false 0.1192	`noul: 0.8808`
urgency (추가 질문)	`[0, 1, 3]`	0: 0.0420 · 1: 0.1142 · 2: 0.8438	`score: 1.8018`

choice

최대 확률의 선택지
confidence + 전체 확률

noul

true의 확률
참·거짓 문장을 생성하지 않음

score

Σ 단계 인덱스 × 확률
최빈 단계와 다를 수 있음

표의 확률은 표시용 반올림. score는 원래 확률로 계산한다.
이 참조 코드의 confidence는 최대 선택지 확률이며, 실제 정답률을 보장하지 않는다.

소스: systemone_answer · systemone

필드 이름은 보관하고, 모델 점수만 값으로 연결한다

```python
# 실제 응답 조립 코드를 단순화한 예시
answers = {}
for q, scores in zip(encoded.questions, logits):
    p = scores.float().softmax(-1).tolist()
    probabilities = dict(zip(q.option_ids, p))
    answers[q.question_id] = systemone_answer(
        questions[q.question_id], probabilities
    )
```

입력에서 보관한 ID + 모델에서 계산한 값

- `team`, `refund` : 입력의 질문 ID
- `billing`, `technical` : 입력의 선택지 ID
- 점수·확률·선택 결과: 모델 출력에서 계산

```json
{
  "answers": {
    "team": {
      "type": "choice",
      "choice": "billing",
      "confidence": 0.8808,
      "probabilities": {
        "billing": 0.8808,
        "technical": 0.1192
      }
    },
    "refund": {
      "type": "noul",
      "noul": 0.8808
    }
  }
}
```

JSON은 프로그램이 조립한다. 모델은 JSON 문자열 대신 선택지 logits를 출력한다.

LoRA와 결정 헤드의 학습률을 분리한다

$$W' = W + \frac{\alpha}{r} BA$$

부분	실습 설정	목적
본체	4-bit 로딩	큰 본체로 바꿔도 같은 설정 유지
LoRA	rank 16, alpha 16, **모든 선형층**	본체의 저랭크 업데이트
LoRA 학습률	`2e-4`	표현 적응
새 결정 헤드 학습률	`1e-4` (Unsloth 기본값)	무작위 초기화된 헤드 학습

0.8B는 16-bit로도 메모리에 충분히 들어간다. 4-bit와 헤드 `1e-4` (기본값, 문서에 근거 없음)는 **확장 실험에서 바꿔 볼 변수**다.

먼저 예측: 학습 전 정확도는 채팅 LLM의 zero-shot 정확도와 비슷할까? → 02의 학습 전 baseline으로 확인한다.

label은 학습할 답, gold는 기준으로 삼은 답

label은 학습·평가에 사용하는 답을 뜻한다.
gold는 기준 답으로 채택한 label을 강조하는 표현이다.

형태	세 선택지의 예시	학습에 전달하는 정보
Hard label / hard gold	`billing` 또는 `[1, 0, 0]`	하나의 정답 · 정답에 1, 나머지에 0
Soft label / soft gold	`[0.7, 0.2, 0.1]`	billing 70% · technical 20% · sales 10%

q: 기준 답의 분포

학습 데이터가 제공하는 목표
이 실습에서는 교사 모델이 만든 soft gold

p: 모델의 예측 분포

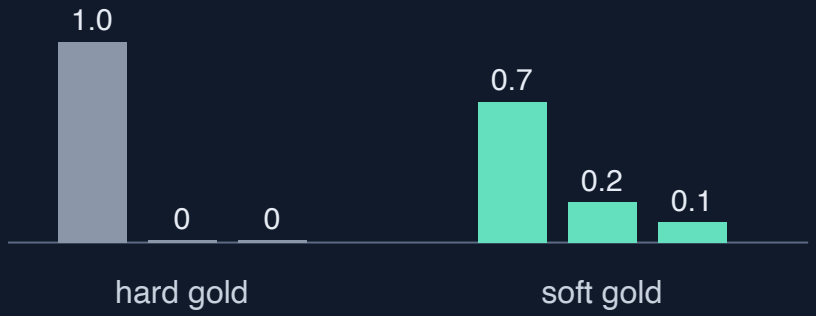
학습 중인 모델이 출력하는 확률
교차 엔트로피로 q를 따라가도록 학습

gold ≠ 완벽한 정답. 교사의 불확실성을 담을 수 있지만, 편향과 오류도 포함한다.

Hard label보다 많은 정보를 담는 soft gold

$$\mathcal{L}_{CE} = -\sum_{k=0}^{K-1} q_k \log p_k$$

교사: 데이터를 만든 약 4B급 LLM. 사례마다 temperature 0.7로 세 번 답하고, 세 분포의 평균이 soft gold `q` 가 된다.

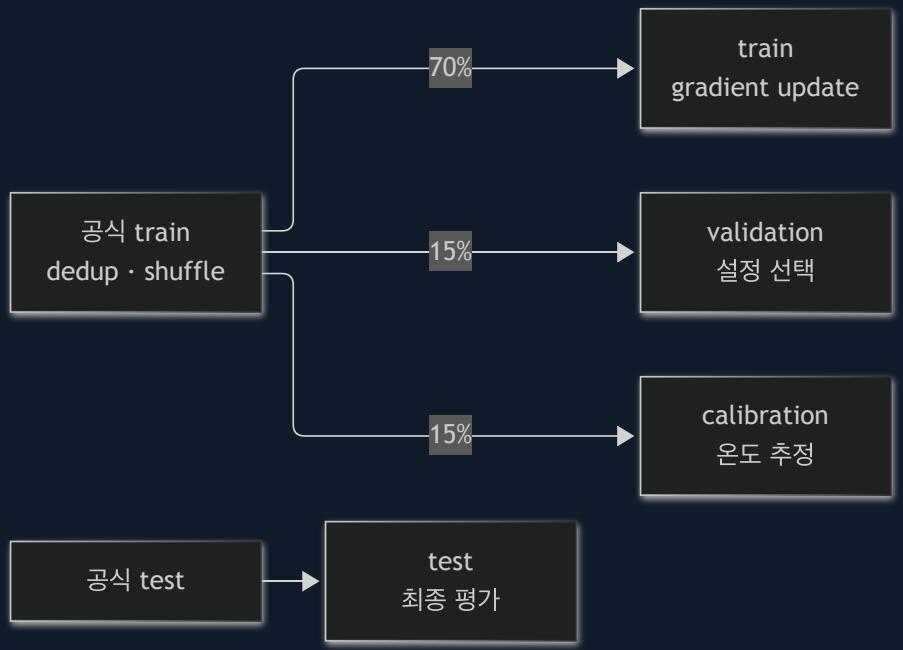


설명용 같은 사례의 세 선택지 정답 분포 q

- 애매한 사례에서 **교사의 불확실성**을 남길 수 있다.
- 애매한 사례에서 과신을 덜 배우게 한다. 그래도 **보정은 calibration 데이터로 따로 확인**한다(Part 3).
- soft gold 역시 교사의 편향과 오류를 포함한다.

typed-decisions 데이터 카드 · DecisionTrainer 구현

데이터는 네 역할로 나누고 섞지 않는다



분할	하지 않는 일
train	최종 성능 주장
validation	gradient update
calibration	본체 학습
test	설정·온도 선택

validation은 설정을 고르며 여러 번 들여다본 데이터다. 그래서 온도는 **아직 보지 않은 calibration**으로 맞춘다.

> 한 state의 질문을 **서로 다른 분할에 넣지 않는다.** 고객·문서 그룹도 고려한다.

PART 03 · MEASUREMENT

맞힌 비율과 확률의 품질

노트북 01을 실행한다 · 약 15분 · 장마다 ▶ **표시된 절**까지 실행

네 가지 지표를 같이 본다

지표	질문	이 실습에서의 정의
Accuracy ↑	가장 큰 확률의 답이 맞나?	hard label 일치율
NLL ↓	정답에 충분한 확률을 주나?	$-\log p_y$ 평균
Brier ↓	분포가 정답과 가까운가?	클래스 제곱오차 **합**의 평균
ECE ↓	확신과 정답률이 맞나?	top-label(고른 답의 확률만 사용), 5 bins

$$\mathrm{ECE} = \sum_{b} \frac{|B_b|}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|$$

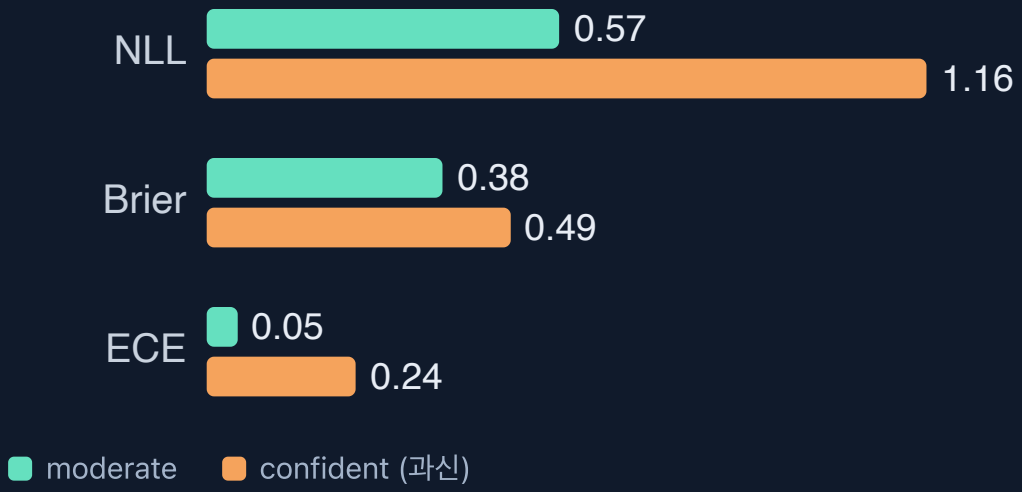
soft gold로 학습해도 accuracy는 **gold의 최댓값 선택지**를 정답으로 본다. Unsloth `evaluate`의 loss는 soft gold CE다. 정의가 다르면 수치를 직접 비교하지 않는다.

실험: 정확도 75%, 확률은 다른 두 모델

```
labels = [0, 1, 0, 1]
moderate = [[.7,.3], [.3,.7], [.7,.3], [.7,.3]]
confident = [[.99,.01], [.01,.99], [.99,.01], [.99,.01]]
```

둘 다 네 개 중 세 개를 맞힌다. **accuracy = 0.75**

토론: 틀린 한 건에 99%를 준 모델의 NLL은 어떻게 될까?

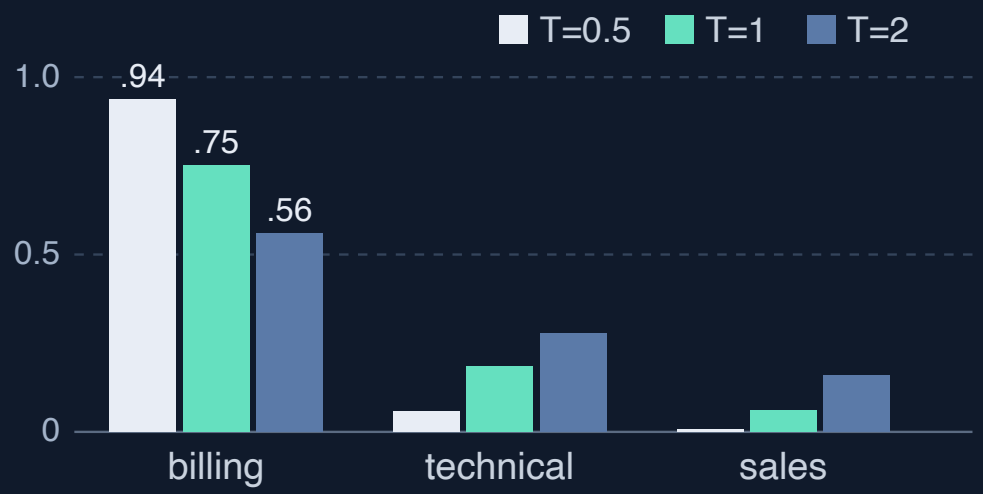


✓ **정상:** 01 §3 출력의 NLL이 0.57 · 1.16으로 차트와 같다

Temperature는 순위를 두고 확신만 조정한다

$$p_k(T) = \frac{\exp(z_k/T)}{\sum_j \exp(z_j/T)}$$

- 생성 LLM의 **샘플링 온도와 다르다.** 학습 후 확률만 조정한다.
- `T > 1` 완만해진다 · `0 < T < 1` 집중된다.
- 양의 스칼라 T는 **argmax를 유지**한다.
- calibration에서 **NLL이 최소인 T**를 골라 test에 고정한다. Unsloth는 질문 타입별로 T를 맞춘다.

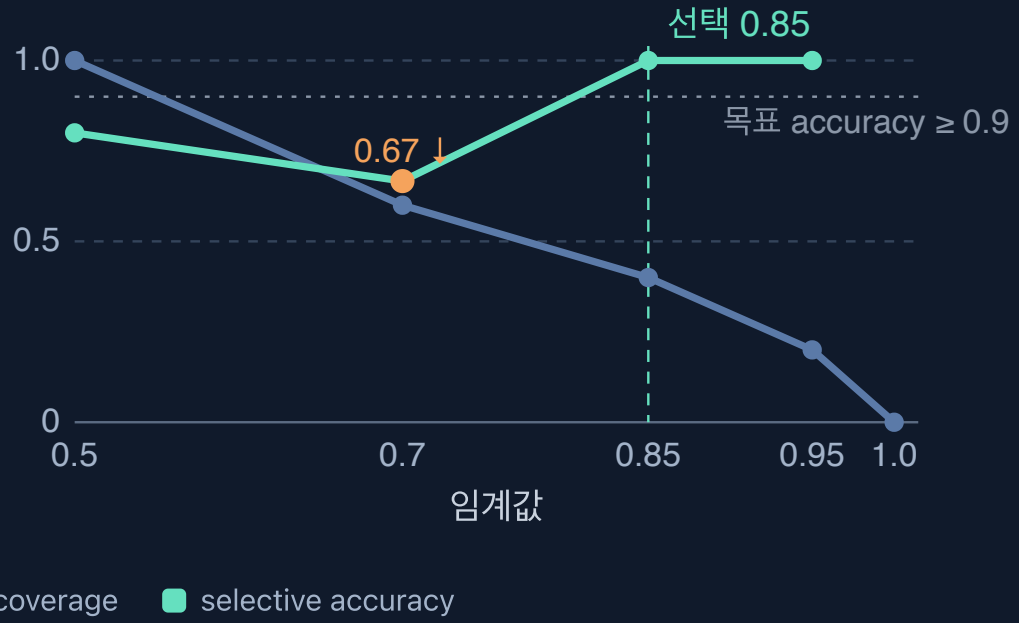


01 §4에서는 calibration 4건으로 **T = 4.0**이 선택된다. 과신을 크게 낮추는 값이다.

임계값을 올리면 자동 처리 범위가 줄어든다

```
if probabilities[chosen_team] >= threshold:
    route_to(chosen_team)
else:
    route_to("human_review")
```

- **coverage**: 자동 처리 비율
- **selective accuracy**: 자동 처리분의 정확도
- 실수 비용·검토 비용은 별도 측정



규칙: 목표 accuracy 0.9(예시 업무 요구)를 넘는 임계값 중 **coverage가 가장 큰 값** → 0.85. 정확도는 단조롭지 않으므로 validation에서 고르고 test에서 확인한다.

PART 04 · TRAINING LAB

GPU 서버에서 실제 학습

노트북 02 · Unsloth + Qwen + LoRA

smoke 설정으로 작게 시작한다

항목	기본 설정
OS / Python	Linux / 3.11 또는 3.12
GPU	단일 NVIDIA CUDA, 16GB 이상을 시작 계획으로 권장
본체	`unsloth/Qwen3.5-0.8B`
smoke	공식 train 240행(→ 70/15/15 분할) / test 40행 / 60 steps
full	전체 데이터 / 2 epochs
입력 길이 / batch	2048 / 1 × accumulation 8

16GB는 실측 최소 사양이 아니다. 실제 peak memory를 노트북에서 기록한다.

✓ **정상:** §1 import에 오류가 없고, §2에서 GPU 이름과 VRAM이 출력된다

재현성을 실행 산출물로 남긴다

```
artifacts/qwen-decisions-<UTC timestamp>/
  run_manifest.json   # model/data revision, seed, splits, settings
  pip-freeze.txt      # 실제 설치 버전
  metrics.json        # 학습 전후, 보정, 최종 test
  adapter/           # LoRA + 결정 헤드 + 보정 온도 → 재로딩 검증용
  reload_check.json  # 재로딩 확률 차이
  merged/            # 16-bit 전체 모델 → Studio 서빙용(Part 5), EXPORT_MERGED
```

모델/데이터 SHA를 고정할 수 있다. **seed 하나만으로 하드웨어 간 완전 재현을 보장하지 않는다.**

데이터 변환이 조용히 실패하지 않도록

```
items, report = FastDecisionModel.build_dataset(rows, tokenizer, model)
print(report)
assert items
assert report["skipped"] == 0
```

신호	대응
`questions` · `gold` 가 JSON 문자열	파싱 후 변환
`skipped > 0`	**멈추고** 정답·선택지·빈 split 수정
`truncated`	기록하고 state 끝의 근거 손실 확인
train/test exact-state 중복	**실행 중단**

✓ **정상:** 모든 split에서 `skipped == 0`, 중복 검사 assert 통과

Trainer에는 train과 validation만 넘긴다

```
args = TrainingArguments(learning_rate=2e-4, ...)  # LoRA 학습률
trainer = DecisionTrainer(
    model=model, processing_class=tokenizer,
    train_dataset=items["train"],
    eval_dataset=items["validation"],
    head_learning_rate=1e-4,                       # 헤드 학습률
    args=args,
)
trainer.train()
```

> calibration과 test는 **Trainer에 넘기지 않는다.**

학습 loss만 내려가면 성공일까? → validation, 보정, test를 함께 확인한다.

✓ **정상:** OOM 없이 시작되고 loss 로그가 찍힌다 → 기다리는 동안 다음 두 장

0.735는 수학적 상한이 아니라 교사 자기 일치도다

0.735

교사의 새 샘플 하나를
나머지로 만든 gold와 비교한 정확도

수학적 상한 ×

항목	typed-decisions
데이터	상태·질문·soft gold로 구성된 업무 데이터
업무	고객 서비스, 청구서, 보안, 에이전트 실행 기록
specialist	이 train으로 학습해 **네 업무만** 답하는 모델
general	이 데이터를 본 적 없이 요청마다 받은 스키마로 답하는 모델

참고 글과 카드는 이 값을 “ceiling”이라 부르지만 **수학적 상한은 아니다.** 0.735를 크게 넘으면 교사의 버릇을 학습했다는 신호다.

데이터 카드: 생성 방식, 기준선, 평가 조건

기사 수치를 실습의 예상 점수로 쓰지 않는다

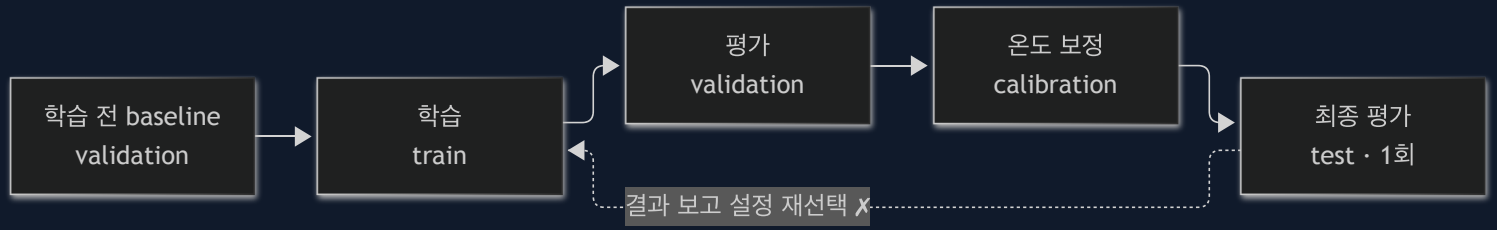
공식 가이드 보고 (Qwen3.5-0.8B)	holdout accuracy, 학습 전 → 후
typed-decisions	36% → 73%
BANKING77 (은행 고객 문의 77개 의도 분류)	7% → 74%

가이드는 학습 전 값을 "대략 우연 수준"이라고만 설명한다. 보고 설정과 이 실습의 데이터 크기·분할·steps는 다르다.

> **보고값은 참고 자료이며 이 강의에서 재현한 결과가 아니다.**

Unsloth 보고 결과 · 확인일 2026-10-13

평가는 validation → 보정 → test 순서다

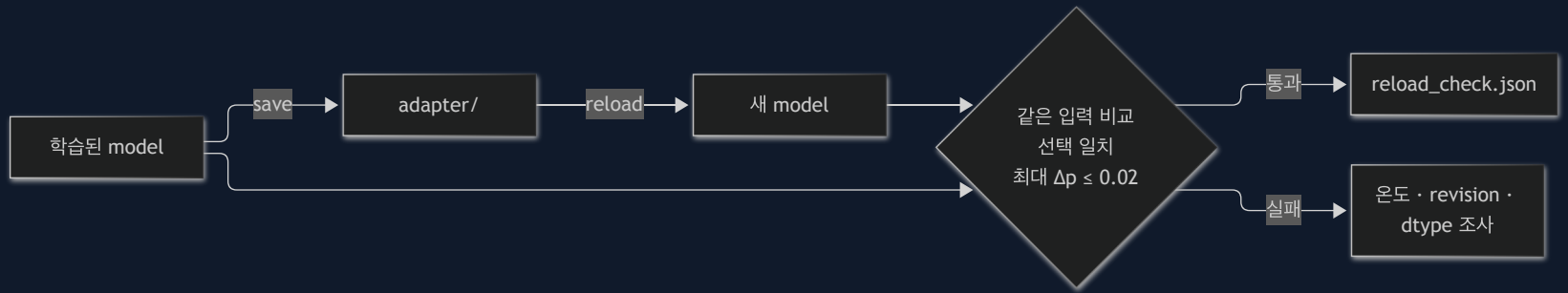



```
# 0. 학습 전 baseline (trainer.train() 전에 실행)
before = FastDecisionModel.evaluate(model, tokenizer, validation_items)
# 1. 학습이 validation에 도움이 되었나?
after = FastDecisionModel.evaluate(model, tokenizer, validation_items)
# 2. 별도 calibration 데이터로 온도를 맞춘다.
FastDecisionModel.calibrate(model, tokenizer, calibration_items)
calibrated = FastDecisionModel.evaluate(model, tokenizer, validation_items)  # 보정 효과
# 3. 고정한 모델의 최종 test 결과는?
final = FastDecisionModel.evaluate(model, tokenizer, test_items)
```

> 최종 test를 보고 다시 설정을 고르면 **그 test도 개발 데이터가 된다.**

저장이 끝났다면, 다시 불러와 비교한다

```python
model.save_pretrained("adapter")
# 기존 trainer/model 참조를 해제하고 CUDA cache를 비운 뒤
model, tokenizer = FastDecisionModel.from_pretrained(
    "adapter", load_in_4bit=True
)
```



저장 전에 `EXPORT_MERGED = True`로 바꾼다(기본값 False). Part 5의 Studio 서빙에 merged 모델이 필요하다.

✓ **정상:** 재로딩 비교 통과. 온도는 `unsloth_decision_config.json`에 함께 저장되고, 0.02는 이 강의가 정한 기준이다

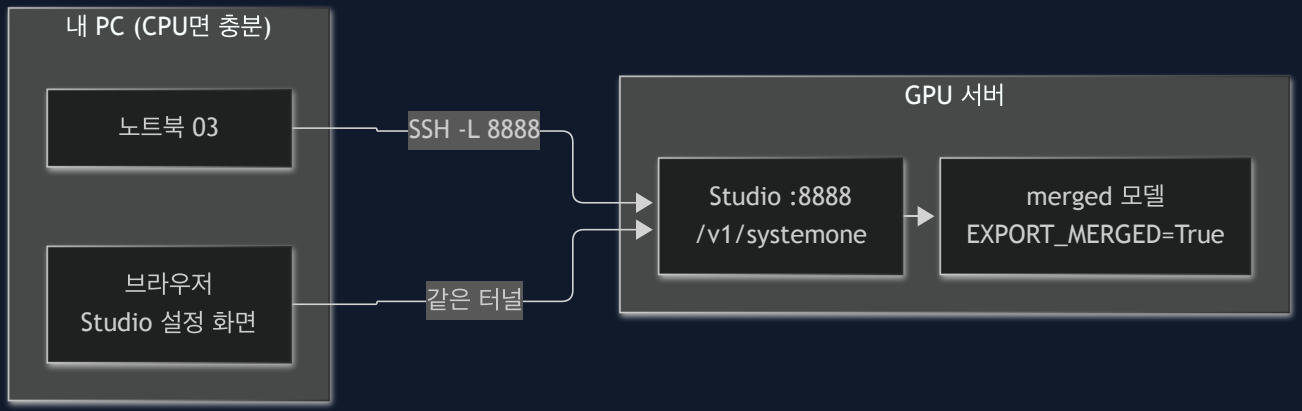
PART 05 · SERVING LAB

로컬 API에 연결하고 검증한다

노트북 03

학습 모델을 Studio로 연결

```
UNSLOTH_SYSTEMONE_MODEL=/absolute/path/to/merged \
  unsloth studio -H 127.0.0.1 -p 8888
```




```
ssh -L 8888:127.0.0.1:8888 user@gpu-server
```

내 PC 브라우저에서 `127.0.0.1:8888` → Settings → API → Decision API → **Serve requests**. 03을 서버의 Jupyter에서 돌리면 터널 없이 바로 연결된다.

API 테스트를 세 층으로 나눈다

층	테스트	모델이 필요한가?
계약	키, 타입, 유한 확률, 합계, 기대 점수	아니오
연결	HTTP, 인증, 선택 모델, 첫 요청	예
품질	독립 라벨 정확도, 보류, 지연시간	예

먼저 예측: 계약 테스트를 통과하면 모델 정확도도 검증된 것일까? 03의 기본값은 `LIVE_API=False` 다.

> 02 모델은 영어 typed-decisions의 네 업무로 짧게 학습했다. 03은 **처음 보는 스키마를 한국어로** 묻는다. 정확도가 낮아도 실습 실패가 아니라 **일반화의 관찰 결과**다.

`score` 응답은 직접 검산할 수 있다

```
probs = answer["probabilities"]
expected = sum(int(level) * p for level, p in probs.items())
assert abs(answer["score"] - expected) < 1e-3
```

검사	통과 조건
choice	선택 키 = 확률 최대 선택지
확률 값	유한하고 `[0, 1]` 범위
확률 키	요청 선택지와 일치
지연시간	첫 요청과 warm request를 따로 기록

6건 smoke test의 p95는 사실상 최대값이다. 성능 보고에는 더 많은 표본이 필요하다.

증상별로 한 번에 하나씩 바꿔 진단한다

증상	먼저 확인할 것
ImportError / API 없음	Unsloth 버전, 커널 재시작, import 순서
GPU OOM	batch, 입력 길이, 모델 크기, 이전 모델 참조
skipped / truncated	gold 형식, 선택지 키, 입력 길이
connection refused / timeout	서버, SSH 터널, 포트, 최초 모델 로딩
401 / 403	access token, Keyless API access(localhost에서 키 생략) 설정
한국어 성능 저하	모델 언어 범위, 한국어 평가 데이터

하나씩 바꾸고 revision·설정·측정값을 함께 남긴다.

확장 실험: 어떤 주장을 검증할까?

일반화

새로운 질문 설명
선택지 순서 변경
영어 → 한국어

운영 품질

threshold별 coverage / 오류 비용
긴 입력과 근거 위치
독립 시간 구간의 calibration

실험 계획 제출: 가설 · 고정 조건 · 바꿀 변수 · 분할 · 지표 · 예상 실패 양상 (제출 방법·기한은 실습 공지)

정답 예: “선택지 순서 변경에 강건하다” → 순서만 바꿔 다시 요청하고, **같은 선택지 키끼리** 확률 차이와 선택 일치율을 잰다.

마무리: 다섯 질문에 답할 수 있는가?

1. `score=1.6`은 등급 2를 고른 것과 어떻게 다른가? 목표 1
2. calibration에 test를 사용하면 왜 문제가 되는가? 목표 2
3. 정확도가 같아도 자동화 임계값의 효과가 다른 이유는? 목표 2
4. `jev-latest`를 로컬 서버에 보내면 어떤 모델이 응답하는가? 목표 4
5. 저장 성공 외에 재로딩 검증이 필요한 이유는? 목표 3

> 시작 전 예측 세 문항에 다시 답하고, 처음 답과 비교한다.

오늘의 핵심과 다음 단계

시작 전 예측	정답
0.9로 답하면 정답률도 90%일까?	**보장되지 않는다.** calibration으로 검증할 성질이다
학습 전 정확도가 낮으면 본체가 약한가?	**아니다.** 새 결정 헤드가 무작위 초기화 상태다
보정 온도를 바꾸면 accuracy가 바뀔까?	**아니다.** argmax는 그대로, 확신만 바뀐다

1. 결정 모델은 질문 스키마를 입력으로 받아, **한 번의 순전파로** 선택지 분포를 낸다.
2. 정답을 맞히는 것(accuracy)과 **확률을 믿을 수 있는 것**(NLL·ECE)은 따로 잰다.
3. 누수 없는 분할 → 보정 → 재로딩 검증까지 해야 **같은 모델이 API에서 답한다**고 말할 수 있다.

> 다음 단계: 확장 실험 계획 제출(실습 공지 참조) · full 설정으로 다시 학습해 smoke 결과와 비교

출처와 실습 파일

- 사용자 참고 글 · PyTorch 한국 사용자 모임
- TypeSafe · Jev와 System One 소개
- Unsloth · 결정 모델 학습 가이드
- Unsloth · 로컬 Decision API
- typed-decisions · 데이터 카드

세부 코드 revision과 추가 출처: **sources.md**

강의 실행: `npm run dev` · 실습: `output/jupyter-notebook/`

문서 확인: 2026-10-13. upstream 구현과 문서는 이후 변경될 수 있다.